



Does a planning tool help LLMs on PDDL tasks?

5 language models • 5 PDDL planning tasks • with vs. without a real planner/validator tool • success rates with 95% confidence intervals

The headline result — simulate

Tracking world state through a plan — the simulate task — simply does not happen without the tool:

0%

no-tools — every model, all 1,500 trials

65–92%

tool merely available ($\geq 9B$)

83–97%

tool + one steering sentence ($\geq 9B$)

solve is nearly as stark: 8–11% without the tool, 63–99% with it. The rest of this deck quantifies that pattern across all five tasks — and what it costs in tokens.

Scorecard — six research questions, six verdicts

RQ	question	verdict	evidence (≥9B, think=off)
RQ0.1	does the tool help validate domain/problem files?	YES	availability +21...+30pp; 3/3 sig-for, 0/3 sig-against
RQ0.2	does the tool help SOLVE?	YES	availability +54...+92pp; 3/3 sig-for, 0/3 sig-against
RQ0.3	does the tool help check a plan?	MIXED	availability -67...+13pp; 1/3 sig-for, 2/3 sig-against
RQ0.4	does the tool help track state (simulate)?	YES	availability +65...+92pp; 3/3 sig-for, 0/3 sig-against
RQ0.5	does the advantage grow with plan length?	YES	validate_plan gap widens +5 → +7 → +27pp
RQ0.6	does the advantage track object count?	NO	validate_problem gap flat (+22 / +23 / +23pp)

What we're testing

PDDL is the standard formal language AI planners use to describe a world, the actions allowed in it, a goal to reach, and step-by-step plans. We give a language model five PDDL jobs:

- ▶ solve — find a plan (a sequence of actions) that reaches the goal
- ▶ validate_domain — check that the file defining the world and its actions is correct PDDL
- ▶ validate_problem — check that the file listing the objects, the start state and the goal is correct PDDL
- ▶ validate_plan — decide whether a given plan actually works (a yes/no verdict)
- ▶ simulate — track how the world changes as a plan is run, one action at a time

The question: do models do these jobs better when we also let them call a real planner/validator program — a “tool” — instead of answering only from their own knowledge?

The three setups we compare

Every job is run in three setups (we call them “arms”), and we never mix their results:

- ▶ no-tools — the model answers on its own, with no external help. This is the baseline.
- ▶ tool available — exactly the same request, but now a real planner/validator is there for the model to call if it chooses to.
- ▶ tool + nudge — the tool is available AND we append one sentence explicitly telling the model to use it (we call this “steered”).

Comparing neighbouring arms answers two separate questions:

- Does simply having the tool help? (no-tools vs tool available — the wording is otherwise identical)
- Or did the model just need to be told to use it? (tool available vs tool + nudge)

In the tables these arms appear as the short codes nt-neut · tl-neut · tl-ster (same order).

How to read the results

- ▶ success rate — how often the model's answer matched the known-correct answer, 0–100%. It is the only score shown; arms are scored separately and never pooled.
- ▶ Error bars (charts) and [low, high] brackets (tables) are Wilson 95% confidence intervals. When two ranges don't overlap, the difference is real rather than noise; a “*” on a gap marks exactly that. Green = the tool helped, red = the tool hurt.
- ▶ “≥9B” — headline conclusions use the larger models (Qwen3.5-9B, Gemma-MoE-26B, Qwen3.6-35B); Qwen3.5-4B is shown for context and the 0.8B failure mode gets its own slide.
- ▶ “think=off” — the model answers directly. think=on (reasons first) is a caveat slide, not the headline.
- ▶ “difficulty bins” (RQ0.5/0.6) — problems split into easy / medium / hard thirds, no-tools vs tool + nudge, to see whether the tool's advantage changes as problems get harder.
- ▶ A contamination re-run on disguised problems (sweep-6) left the no-tools baseline unchanged (footnote).

Why the verdicts count direction — signed significance

A gap counts toward a YES only when the two arms' 95% intervals are disjoint AND the gap points the hypothesised way (the tool helps). Significant gaps pointing the other way are tallied separately as significant-AGAINST — they are findings, not noise.

- ▶ The case that makes this matter: on `validate_plan`, merely making the tool available moves Gemma-MoE-26B by -67pp — significant, and AGAINST the tool (it stops answering).
- ▶ A sign-blind test would count that gap as evidence the tool 'has an effect' and read RQ0.3 as YES. The signed rule reads it as MIXED — which is what the data actually says.
- ▶ The build asserts the computed signed verdicts equal the locked scorecard, so the deck cannot silently drift from this rule.

RQ0.1 — validate_domain + validate_problem

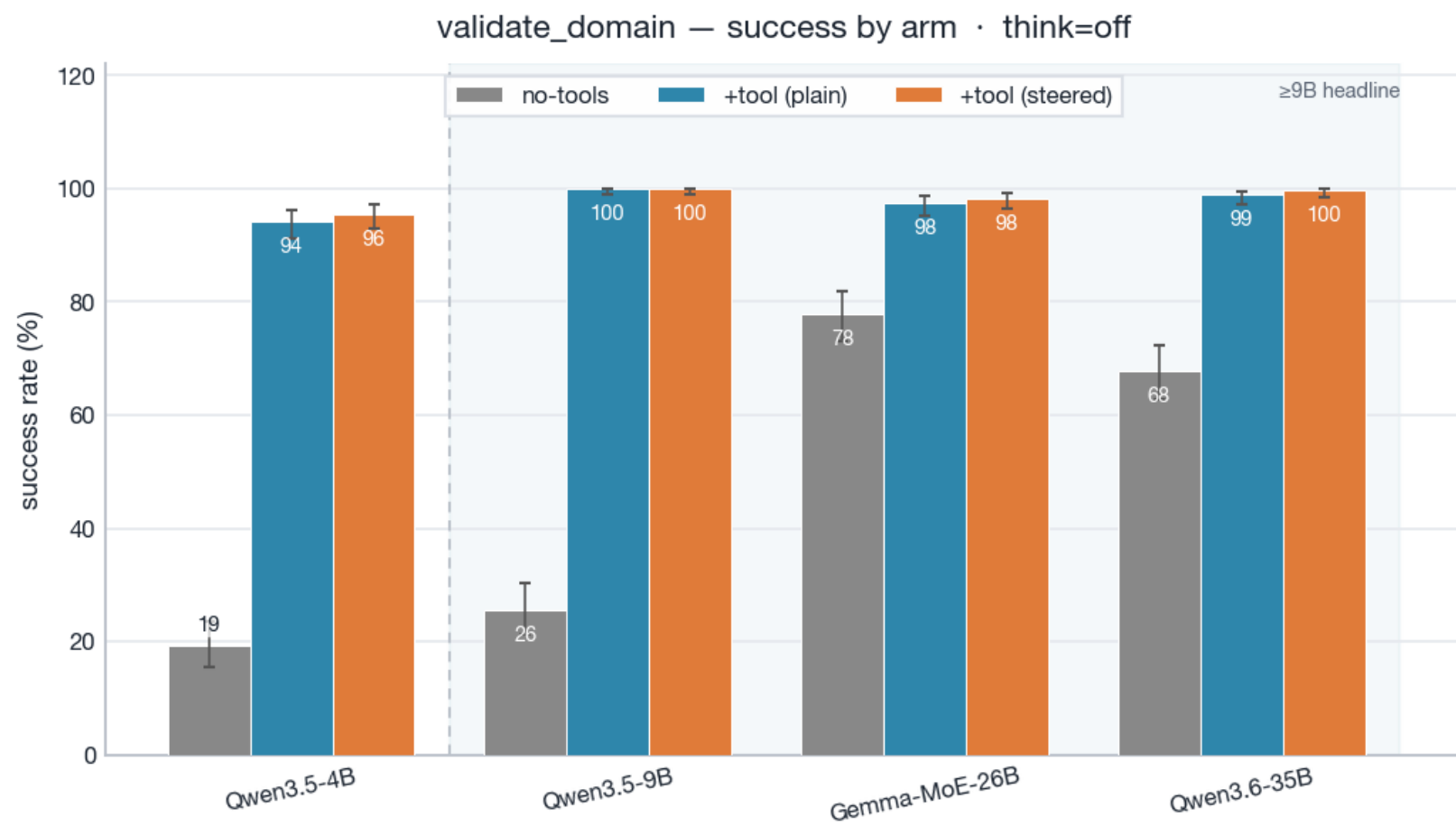
YES

Does giving the model a validation tool help it validate PDDL domains and problems? (validate_domain + validate_problem)

- ▶ Availability (validate_problem, $\geq 9B$): gap +21...+30pp; 3/3 favorable-significant, 0/3 significant-against (95% CIs).
- ▶ Steering (validate_problem, $\geq 9B$): gap -5...+1pp; 0/3 favorable-significant.

Caveat: at 0.8B the availability gap REVERSES on validate_problem (-25pp) — the smallest model mishandles the tool (see the small-model slide). YES holds from 4B up.

RQ0.1 — Evidence: validate_domain success by arm

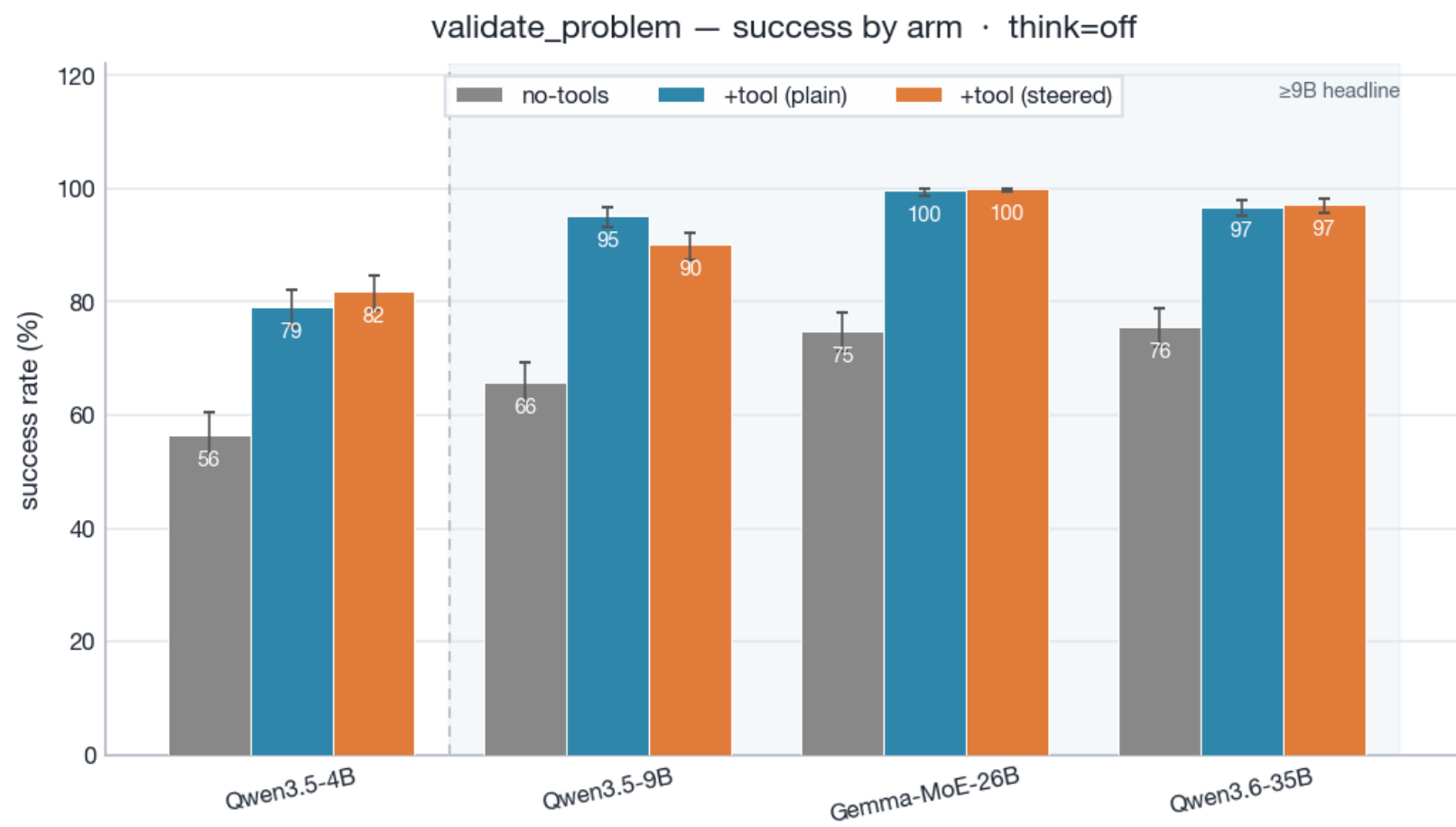


Grey=no-tools, blue=+tool(plain), orange=+tool(steered). Whiskers=Wilson 95% CI. Shaded band marks the ≥9B headline set. 0.8B is excluded (own caveat slide).

RQ0.1 — validate_domain success table (rate [Wilson 95%], * = CI-disjoint)

model	no-tools	+tool(plain)	+tool(steered)	avail Δ	steer Δ
Qwen3.5-4B	19 [15,24]	94 [91,96]	96 [93,97]	+75*	+1
Qwen3.5-9B	26 [21,30]	100 [99,100]	100 [99,100]	+74*	+0
Gemma-MoE-26B	78 [73,82]	98 [95,99]	98 [96,99]	+20*	+1
Qwen3.6-35B	68 [63,72]	99 [97,100]	100 [98,100]	+31*	+1

RQ0.1 — Evidence: validate_problem success by arm



Grey=no-tools, blue=+tool(plain), orange=+tool(steered). Whiskers=Wilson 95% CI. Shaded band marks the ≥9B headline set. 0.8B is excluded (own caveat slide).

RQ0.1 — validate_problem success table (rate [Wilson 95%], * = CI-disjoint)

model	no-tools	+tool(plain)	+tool(steered)	avail Δ	steer Δ
Qwen3.5-4B	56 [53,60]	79 [76,82]	82 [79,85]	+23*	+3
Qwen3.5-9B	66 [62,69]	95 [93,97]	90 [88,92]	+30*	-5*
Gemma-MoE-26B	75 [71,78]	100 [99,100]	100 [99,100]	+25*	+0
Qwen3.6-35B	76 [72,79]	97 [95,98]	97 [96,98]	+21*	+1

RQ0.2 — solve

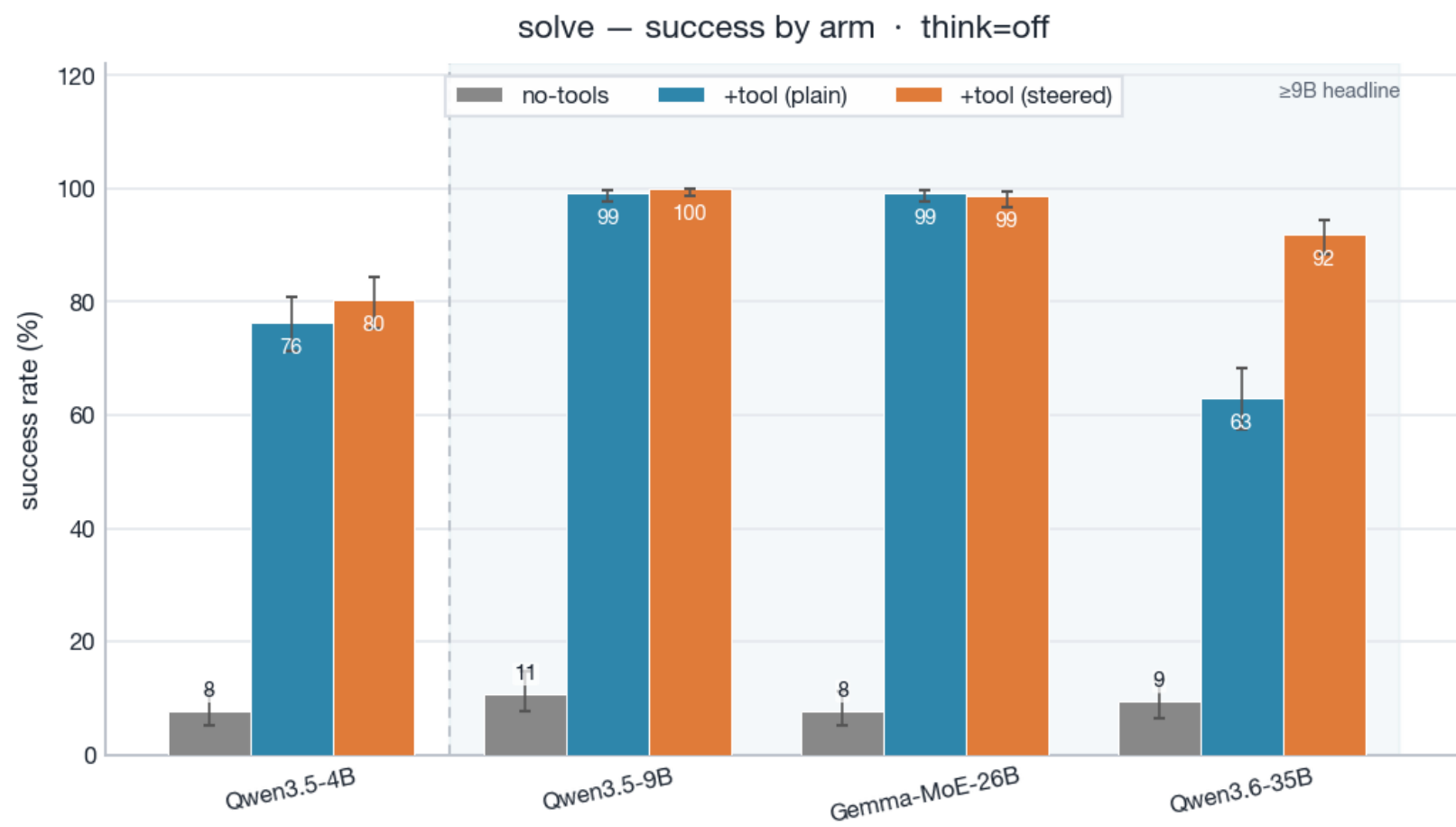
YES

Does giving the model a planning tool help it SOLVE PDDL problems? (solve)

- ▶ Availability (solve, $\geq 9B$): gap +54...+92pp; 3/3 favorable-significant, 0/3 significant-against (95% CIs).
- ▶ Steering (solve, $\geq 9B$): gap -1...+29pp; 1/3 favorable-significant.

Decisive: no-tools is floored (~8–11%); +tool lifts $\geq 9B$ to 63–99%. Steering adds a large further lift where plain left headroom (Qwen3.6-35B +29pp).

RQ0.2 — Evidence: solve success by arm



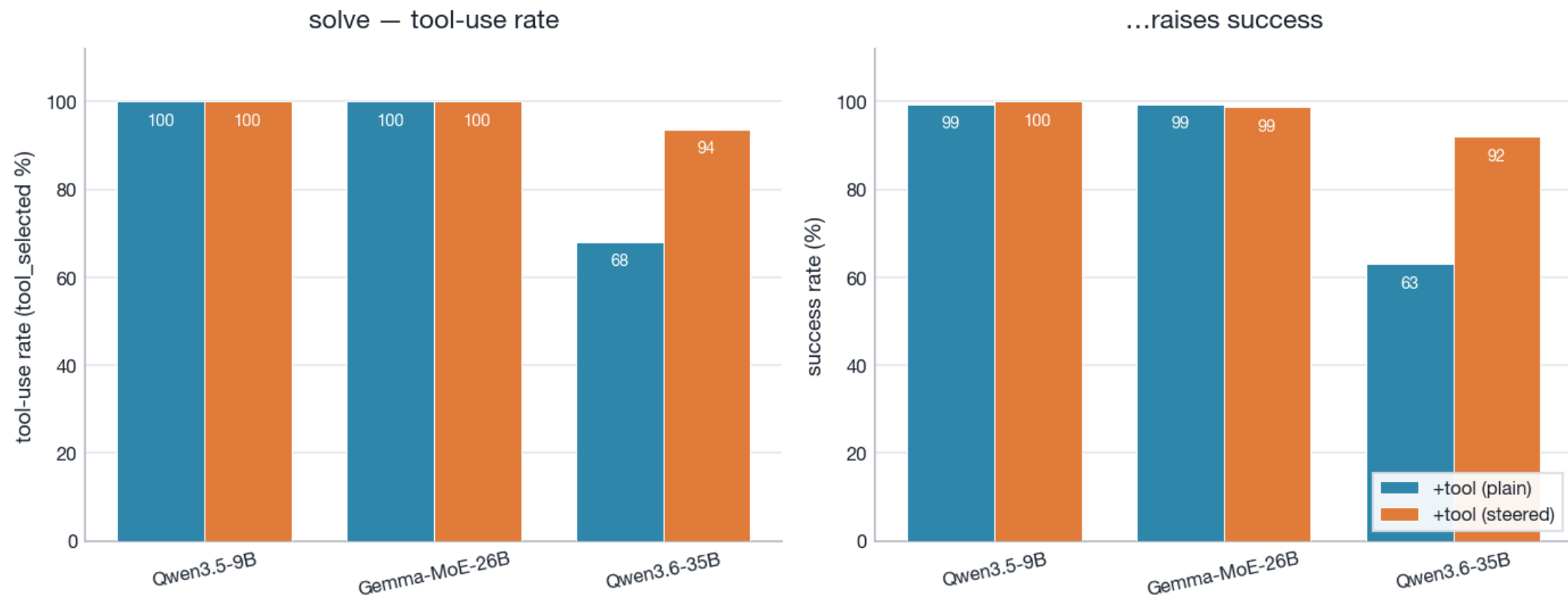
Grey=no-tools, blue=+tool(plain), orange=+tool(steered). Whiskers=Wilson 95% CI. Shaded band marks the ≥9B headline set. 0.8B is excluded (own caveat slide).

RQ0.2 — solve success table (rate [Wilson 95%], * = CI-disjoint)

model	no-tools	+tool(plain)	+tool(steered)	avail Δ	steer Δ
Qwen3.5-4B	8 [5,11]	76 [71,81]	80 [75,84]	+69*	+4
Qwen3.5-9B	11 [8,15]	99 [98,100]	100 [99,100]	+89*	+1
Gemma-MoE-26B	8 [5,11]	99 [98,100]	99 [97,99]	+92*	-1
Qwen3.6-35B	9 [7,13]	63 [57,68]	92 [88,95]	+54*	+29*

RQ0.2 — Mechanism: steering raises tool-calling → raises success

solve mechanism: steering raises tool-calling, which raises success · ≥9B, think=off



solve, ≥9B. Left: tool-use rate (tool_selected) plain vs steered. Right: success. Where +tool(plain) under-calls the tool, steering raises tool-calling, and success rises with it — the tool's value is gated on the model actually invoking it.

RQ0.3 — validate_plan

MIXED

Does giving the model a validation tool help it CHECK whether a plan is valid? (validate_plan)

- ▶ Availability (validate_plan, ≥9B): gap -67...+13pp; 1/3 favorable-significant, 2/3 significant-against (95% CIs).
- ▶ Steering (validate_plan, ≥9B): gap +3...+72pp; 3/3 favorable-significant.

MIXED: the model alone is already strong on validate_plan (75–90%). At +tool(plain) the availability gap is significant-AGAINST for Gemma-MoE (–67pp: it stops answering) and Qwen3.6-35B (–9pp); only Qwen3.5-9B is favorable. Steering RECOVERS and beats no-tools (Gemma 21→93%), but the net tool benefit over a strong baseline is small/mixed.

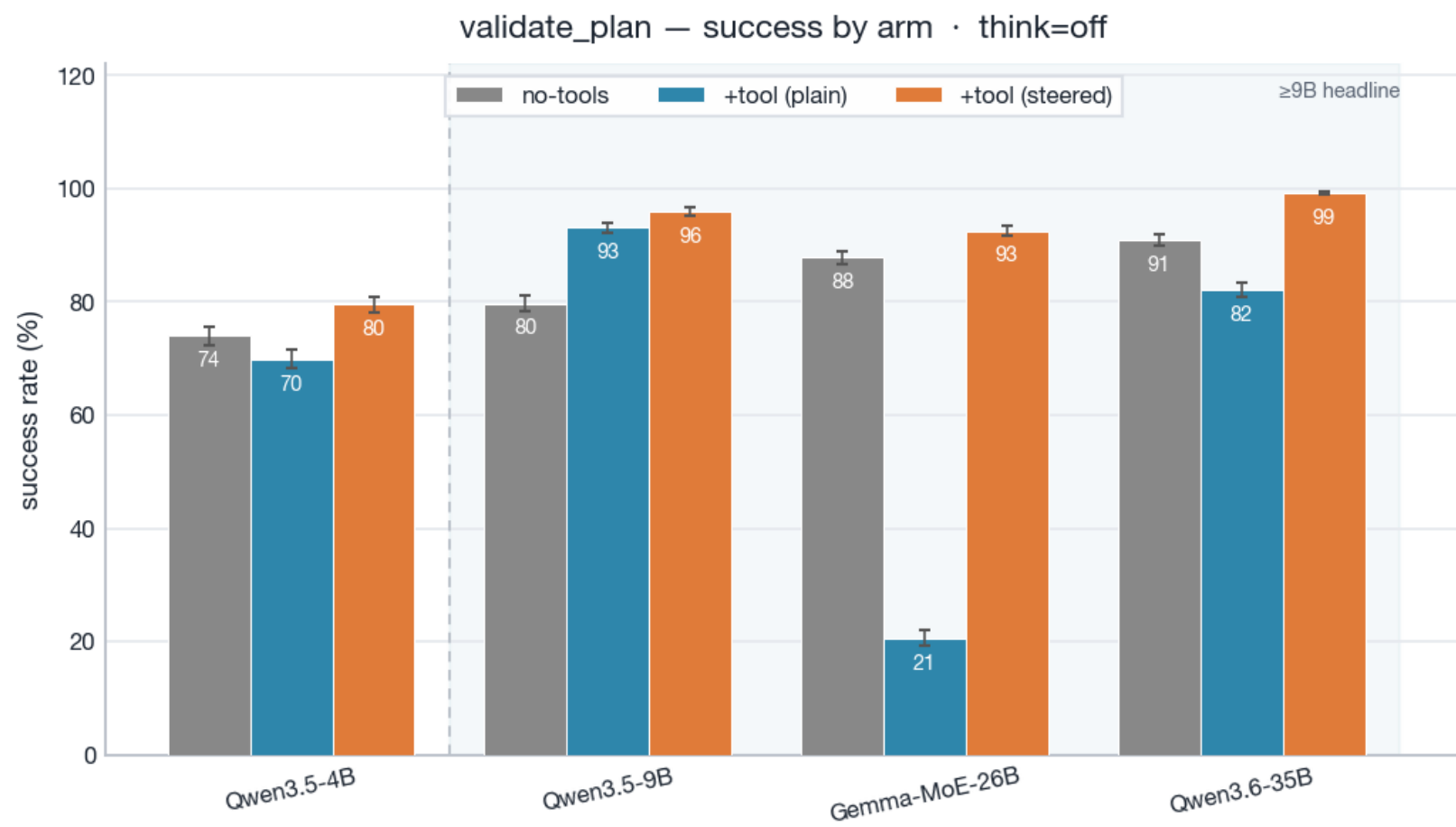
Reading guide — the low +tool(plain) bar is silence, not error

On validate_plan the +tool(plain) arm often produces NO verdict at all — the model fails to call the tool and answers nothing. When it does answer, it is near-perfect:

- ▶ Gemma-MoE-26B — raw success 21%, but it only produces a verdict on 21% of trials; when it DOES answer, accuracy is 99% (false-positives: 5).
- ▶ Qwen3.5-0.8B — raw success 23%, but it only produces a verdict on 34% of trials; when it DOES answer, accuracy is 68% (false-positives: 2).

→ So the collapse is a tool-CALLING failure, not a verdict failure — and steering, which raises tool-calling, repairs it (mechanism slide after the evidence).

RQ0.3 — Evidence: validate_plan success by arm



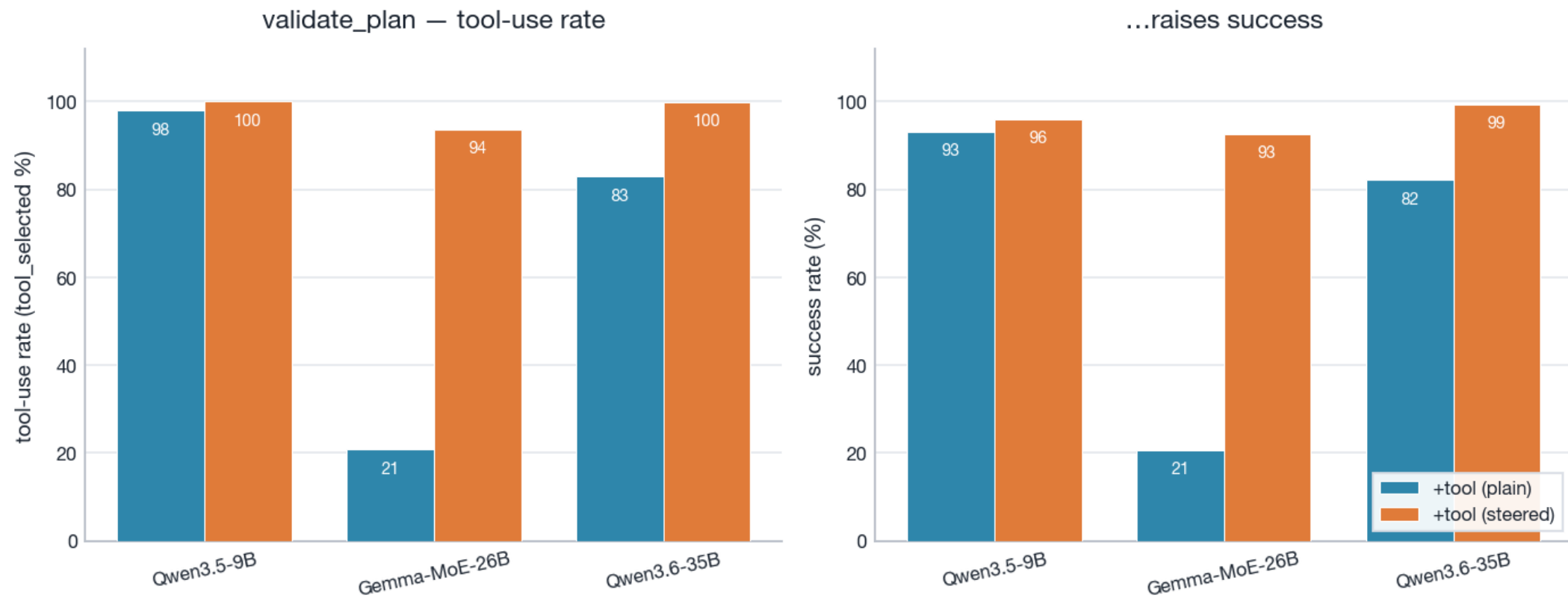
Grey=no-tools, blue=+tool(plain), orange=+tool(steered). Whiskers=Wilson 95% CI. Shaded band marks the ≥9B headline set. 0.8B is excluded (own caveat slide).

RQ0.3 — validate_plan success table (rate [Wilson 95%], * = CI-disjoint)

model	no-tools	+tool(plain)	+tool(steered)	avail Δ	steer Δ
Qwen3.5-4B	74 [72,76]	70 [68,72]	80 [78,81]	-4*	+10*
Qwen3.5-9B	80 [78,81]	93 [92,94]	96 [95,97]	+13*	+3*
Gemma-MoE-26B	88 [87,89]	21 [19,22]	93 [92,93]	-67*	+72*
Qwen3.6-35B	91 [90,92]	82 [81,83]	99 [99,100]	-9*	+17*

RQ0.3 — Mechanism: steering raises tool-calling → raises success

validate_plan mechanism: steering raises tool-calling, which raises success · ≥9B, think=off



validate_plan, ≥9B. Left: tool-use rate (tool_selected) plain vs steered. Right: success. Where +tool(plain) under-calls the tool, steering raises tool-calling, and success rises with it — the tool's value is gated on the model actually invoking it.

RQ0.4 — simulate

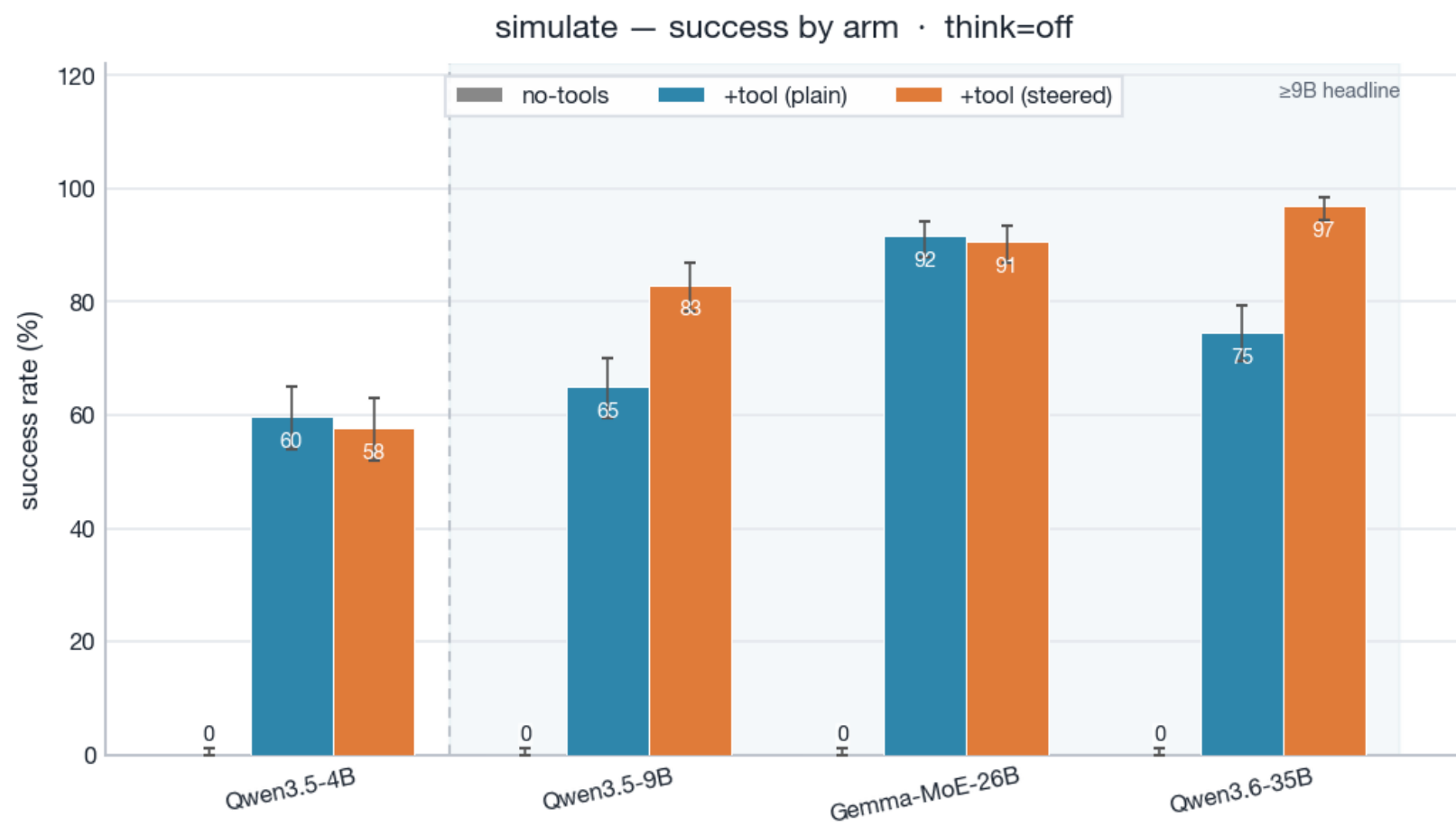
YES

Does giving the model a simulation tool help it TRACK state (simulate a plan)? (simulate)

- ▶ Availability (simulate, $\geq 9B$): gap +65...+92pp; 3/3 favorable-significant, 0/3 significant-against (95% CIs).
- ▶ Steering (simulate, $\geq 9B$): gap -1...+22pp; 2/3 favorable-significant.

Decisive: no-tools is 0% everywhere (state-tracking by hand fails); +tool reaches 65–92% on $\geq 9B$, with steering adding +18–22pp.

RQ0.4 — Evidence: simulate success by arm



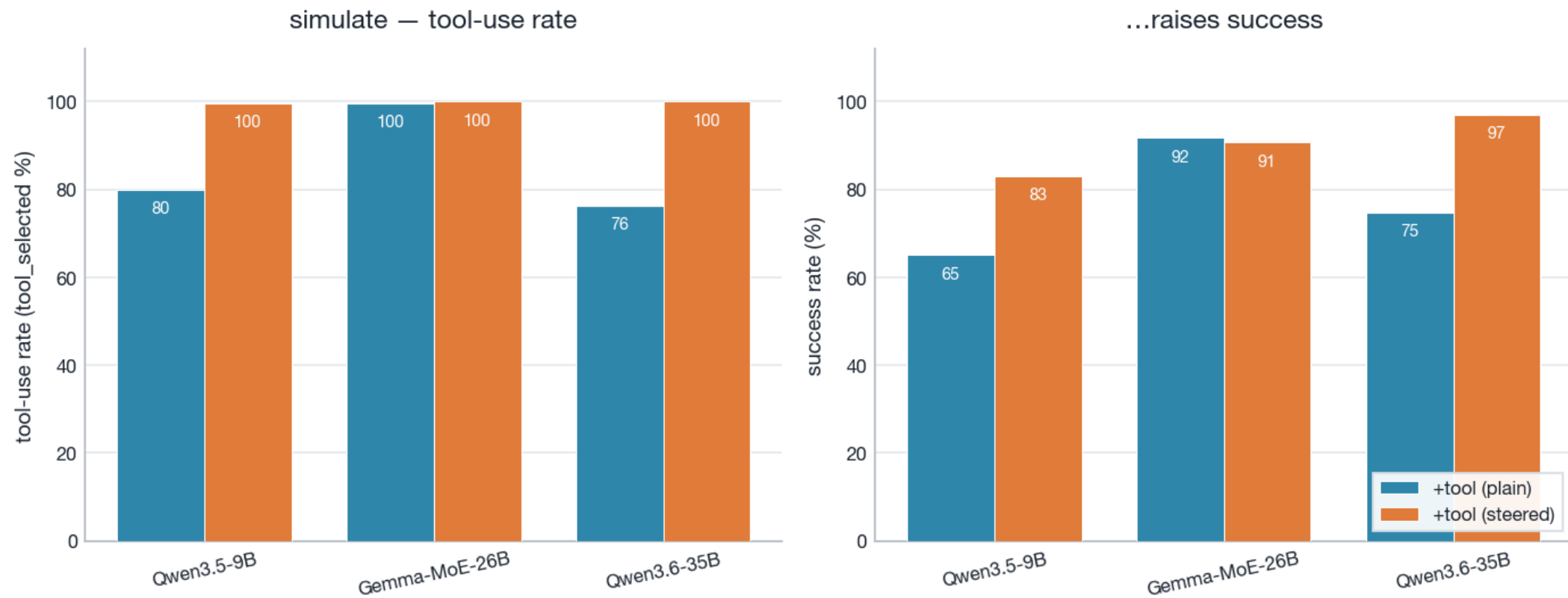
Grey=no-tools, blue=+tool(plain), orange=+tool(steered). Whiskers=Wilson 95% CI. Shaded band marks the ≥9B headline set. 0.8B is excluded (own caveat slide).

RQ0.4 — simulate success table (rate [Wilson 95%], * = CI-disjoint)

model	no-tools	+tool(plain)	+tool(steered)	avail Δ	steer Δ
Qwen3.5-4B	0 [0,1]	60 [54,65]	58 [52,63]	+60*	-2
Qwen3.5-9B	0 [0,1]	65 [59,70]	83 [78,87]	+65*	+18*
Gemma-MoE-26B	0 [0,1]	92 [88,94]	91 [87,93]	+92*	-1
Qwen3.6-35B	0 [0,1]	75 [69,79]	97 [94,98]	+75*	+22*

RQ0.4 — Mechanism: steering raises tool-calling → raises success

simulate mechanism: steering raises tool-calling, which raises success · ≥9B, think=off



simulate, ≥9B. Left: tool-use rate (tool_selected) plain vs steered. Right: success. Where +tool(plain) under-calls the tool, steering raises tool-calling, and success rises with it — the tool's value is gated on the model actually invoking it.

Small-model caveat — Qwen3.5-0.8B mishandles the tool

task	no-tools	+tool(plain)	+tool(steered)	avail Δ	steer Δ
solve	0 [0,1]	12 [9,16]	14 [11,18]	+12*	+2
validate_domain	81 [76,85]	88 [84,91]	89 [86,92]	+7	+2
validate_problem	51 [47,55]	26 [23,30]	18 [16,22]	-25*	-8*
validate_plan	50 [48,52]	23 [22,25]	25 [23,26]	-27*	+2
simulate	0 [0,1]	6 [4,10]	3 [1,5]	+6*	-4

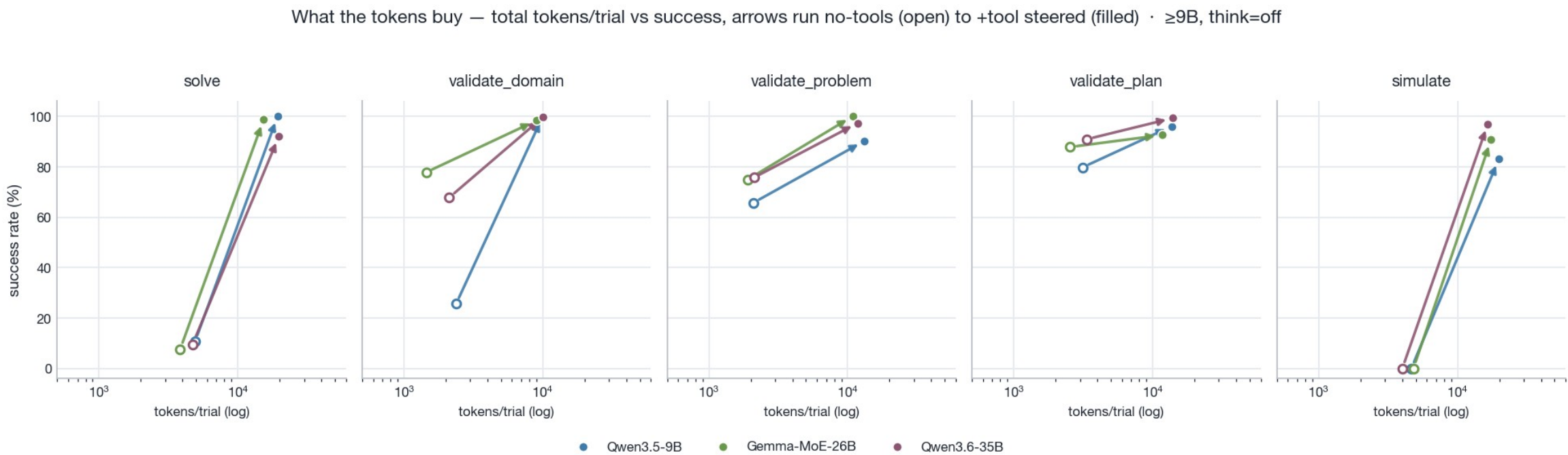
What do tools cost? — token consumption

Token cost = TOTAL tokens a trial consumes — input (prompt) + output (completion), summed across the model's turns. The whole bill, not just generated text. Scope: $\geq 9B$, think=off.

- ▶ Tools raise per-trial consumption $\sim 4\text{--}15\times$ — but consumption alone is half the story: the next figure holds cost and success in one view so neither number is quoted without the other.
- ▶ Quality-adjusted efficiency is cost-of-pass = total tokens $\div$ correct answers: every token burned on a failed attempt is charged to the successes. LOWER is better.

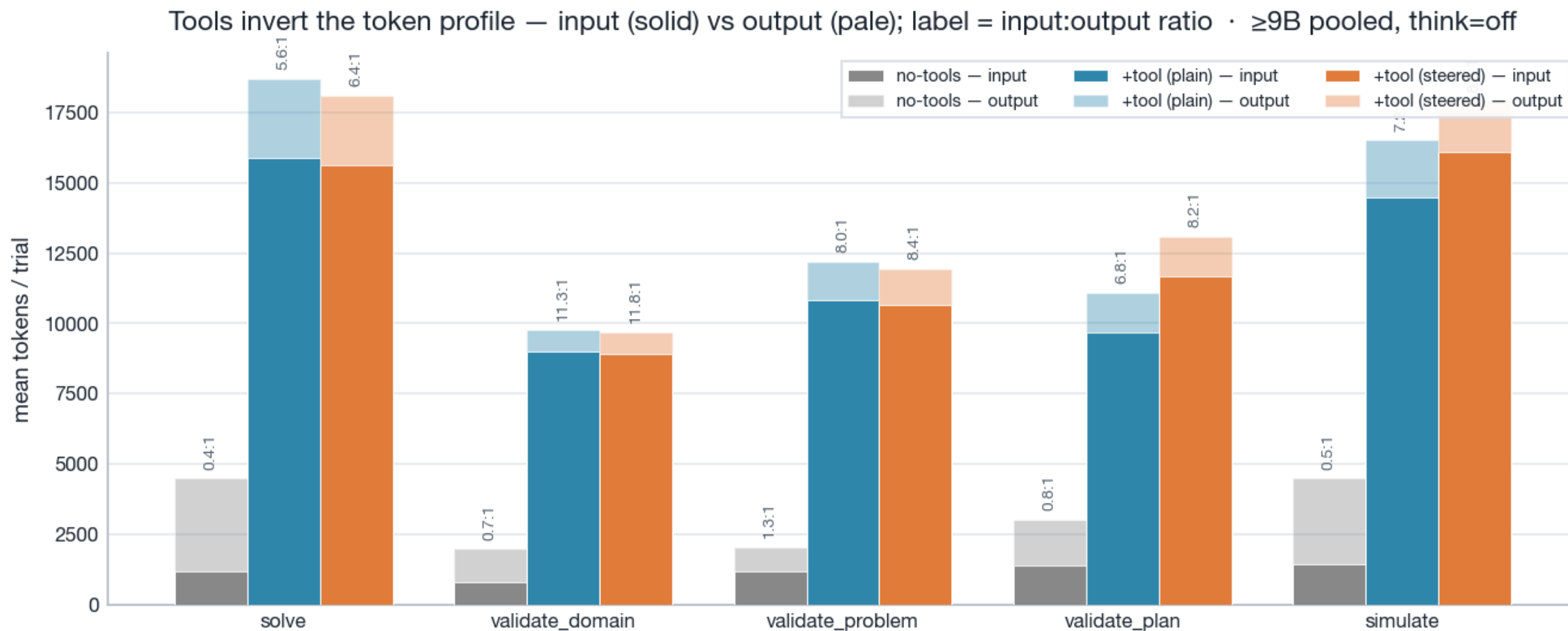
→ Caveats: (1) prefix caching ($\sim 90\%$ on this roster) makes the re-sent tool INPUT cheap in server compute — but it is still real context-budget consumption, and tool OUTPUTS across turns are novel/uncached; the raw total is the honest accounting number. (2) Output is right-censored at the 8,192-token cap with arm-dependent truncation (evidence slide at the end of this section). (3) A completion-only view flatters tools — kept as a labelled secondary lens in the backup.

What the tokens buy — pay ~4–15× per trial, buy +5...+100pp



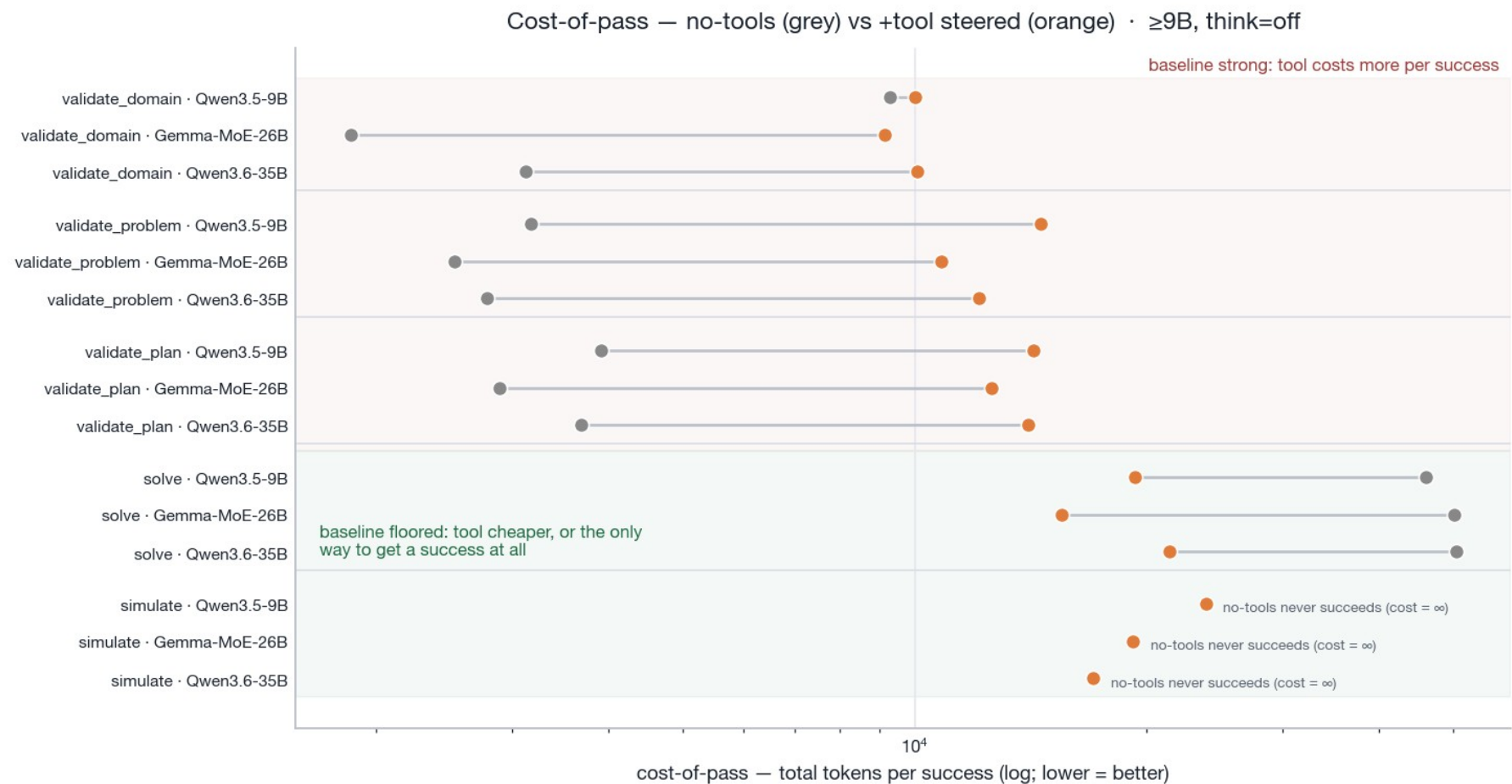
Each arrow is one ≥9B model on one task, from no-tools ○ to +tool(steered) ● — x = mean total tokens/trial (log), y = success. Every arrow points up-and-right: the tool always costs more per trial; the success it buys is decisive where the baseline is floored (solve, simulate) and modest where the baseline is already strong (validate_*). Per-model tables in the backup.

Tools invert the token profile — the cost is input, not output



Mean input (solid) vs output (pale) tokens per trial, ≥9B pooled. no-tools is output-heavy (~0.3:1 input:output — it reasons in the open); the tool arms are input-heavy (~5:1) because tool schemas + tool outputs are re-fed every turn. That re-fed input is the real token cost of tools — an output-only comparison hides it entirely, which is why the headline metric is the total.

Cost-of-pass — the tool pays for itself exactly where the model can't do the task



Total tokens per success (log; lower = better), no-tools ○ vs +tool(steered) ●, bootstrap point estimates. Where the baseline is strong (validate_*) the tool is ~3–11× costlier per success; where the baseline is floored, the tool is ~3× cheaper (solve) or the only source of successes at all (simulate — no-tools never succeeds, so its cost-of-pass is infinite).

Why efficiency moves — (tokens/trial) ÷ (more often right) = cost-per-success

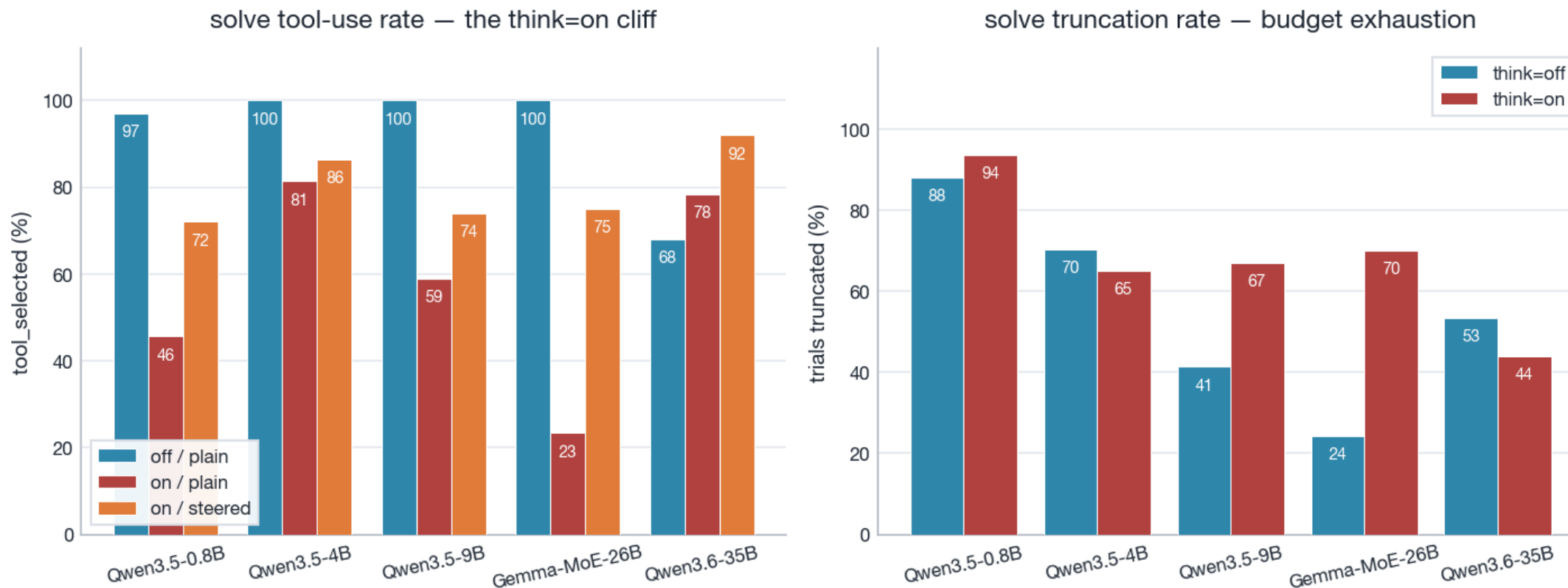
task	model	tokens/trial ×	more often right ×	= cost-per-success ×
solve	Qwen3.5-4B	4.4× costlier	10.5× more right	0.4× cheaper
	Qwen3.5-9B	3.9× costlier	9.4× more right	0.4× cheaper
	Gemma-MoE-26B	4.0× costlier	12.9× more right	0.3× cheaper
	Qwen3.6-35B	4.2× costlier	9.9× more right	0.4× cheaper
validate_domain	Qwen3.5-4B	11.0× costlier	5.0× more right	2.2× costlier
	Qwen3.5-9B	4.2× costlier	3.9× more right	1.1× costlier
	Gemma-MoE-26B	6.2× costlier	1.3× more right	4.9× costlier
	Qwen3.6-35B	4.7× costlier	1.5× more right	3.2× costlier
validate_problem	Qwen3.5-4B	14.9× costlier	1.4× more right	10.3× costlier
	Qwen3.5-9B	6.3× costlier	1.4× more right	4.6× costlier
	Gemma-MoE-26B	5.7× costlier	1.3× more right	4.3× costlier
	Qwen3.6-35B	5.6× costlier	1.3× more right	4.4× costlier
validate_plan	Qwen3.5-4B	8.7× costlier	1.1× more right	8.1× costlier
	Qwen3.5-9B	4.4× costlier	1.2× more right	3.6× costlier
	Gemma-MoE-26B	4.6× costlier	1.1× more right	4.3× costlier
	Qwen3.6-35B	4.2× costlier	1.1× more right	3.8× costlier

Are the token numbers comparable? — truncation, turns, latency proxy

task	arm	truncated %	mean turns	output tok/trial	total tok/trial
solve	no-tools	16%	1.0	3.3k	4.5k
	+tool (plain)	40%	2.7	2.8k	19k
	+tool (steered)	34%	2.7	2.5k	18k
validate_domain	no-tools	4%	1.0	1.2k	2.0k
	+tool (plain)	0%	2.0	792	9.8k
	+tool (steered)	0%	2.0	757	9.7k
validate_problem	no-tools	2%	1.0	871	2.0k
	+tool (plain)	1%	2.1	1.4k	12k
	+tool (steered)	0%	2.1	1.3k	12k
validate_plan	no-tools	1%	1.0	1.6k	3.0k
	+tool (plain)	7%	1.7	1.4k	11k
	+tool (steered)	8%	2.0	1.4k	13k
simulate	no-tools	29%	1.0	3.0k	4.5k
	+tool (plain)	67%	2.0	2.0k	17k
	+tool (steered)	73%	2.2	1.8k	18k

Caveat — think=on is a decode-budget cliff (Gemma-MoE-26B & 9B hardest)

Reasoning-mode budget caveat: think=on collapses tool-calling — Gemma-MoE-26B & 9B hit hardest (budget, not ability)



Left: solve tool-use rate across budget states — neutral think=on collapses tool-calling vs think=off, worst on Gemma-MoE-26B (100→23) and Qwen3.5-9B (100→59), NOT strictly the smallest model (0.8B 97→46); steering partly restores it. Right: solve truncation rate spikes under think=on. The headline uses think=off; the think=on degradation is budget exhaustion, not ability.

RQ0.5 — does the advantage track difficulty?

YES

Does the planning tool's advantage CHANGE as instances get harder (longer reference / plan length)?

YES for validate_plan — the advantage GROWS with plan length.

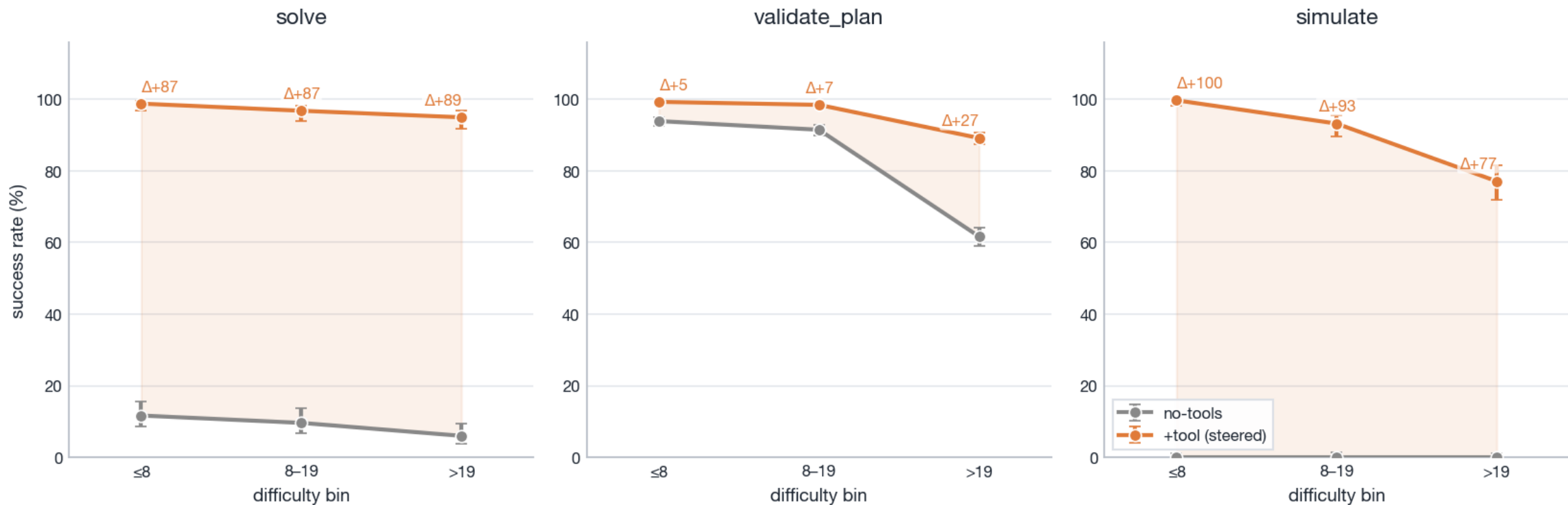
Headroom case (validate_plan) gap by bin: ≤ 8 : $\Delta+5 \rightarrow$ 8–19: $\Delta+7 \rightarrow$ >19 : $\Delta+27$.

- ▶ validate_plan is the headroom-gated case (both arms have room to move): the no-tools – tool gap widens from +5pp (short plans) to +27pp (long plans) as no-tools degrades while the tool holds. This is the generalisation claim: the harder the instance, the more the tool is worth.
- ▶ solve and simulate are framed as tool-arm robustness: no-tools is floored (~ 0 –12%) at every length, so the ~ 87 –99pp gap is large but does not 'grow' — there is no no-tools headroom to lose.

Phase-2 is headroom-gated: an advantage can only be seen to CHANGE where both arms have room to move. $\geq 9B$, think=off, no-tools vs +tool(steered).

RQ0.5 — Evidence

RQ0.5 — does the tool advantage change with PLAN LENGTH? (no-tools vs +tool steered, $\geq 9B$, think=off)



Per difficulty bin: no-tools (grey) vs +tool steered (orange), Wilson 95% CIs; Δ = tool - no-tools over each bin (shaded = the advantage).

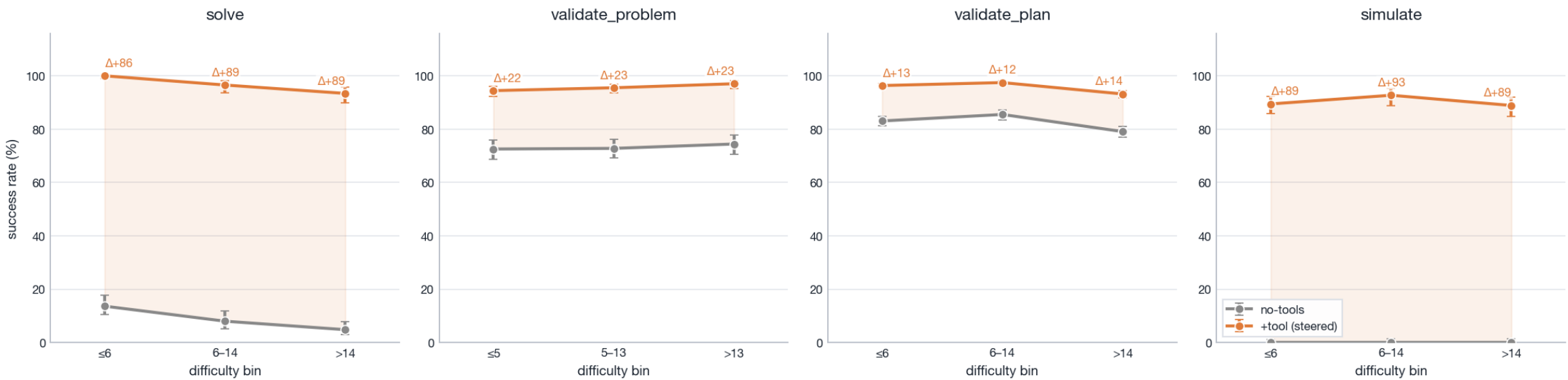
RQ0.5 — difficulty-binned success (no-tools vs +tool steered)

task	bin	no-tools	+tool(steered)	Δ (pp)	n/arm
solve	≤ 8	12 [9,16]	99 [97,100]	+87	324
	8-19	10 [7,14]	97 [94,98]	+87	279
	>19	6 [4,9]	95 [92,97]	+89	297
validate_plan	≤ 8	94 [93,95]	99 [99,100]	+5	1620
	8-19	91 [90,93]	98 [98,99]	+7	1395
	>19	62 [59,64]	89 [87,91]	+27	1485
simulate	≤ 8	0 [0,1]	100 [98,100]	+100	324
	8-19	0 [0,1]	93 [90,96]	+93	279
	>19	0 [0,1]	77 [72,82]	+77	297

RQ0.6 — object count does not move the advantage

NO

RQ0.6 — does the tool advantage change with OBJECT COUNT? (no-tools vs +tool steered, ≥9B, think=off)



Headroom case (validate_problem): gap ≤5: Δ+22 → 5-13: Δ+23 → >13: Δ+23pp — essentially flat; the other tasks are also flat (no-tools floored or already separated). Object count is not a difficulty axis the tool's advantage tracks. Full bin table in the backup.

Robustness footnote — contamination probe (sweep-6)

- ▶ A separate anonymised-corpus sweep (sweep-6) re-ran the matrix on structurally-identical domains with every surface name renamed, to test memorisation.
- ▶ Headline (think=off): the clean no-tools-neutral probe is near-null — $\Delta(\text{canonical} - \text{anon}) \text{ success} \leq 1.3\text{pp mean } |\Delta|$, zero CI-disjoint task cells. No broad train-set contamination of the pure-model baseline where it has headroom.
- ▶ Implication for this deck: the no-tools baselines that the tool's value is measured against are not inflated by memorisation, so the availability gaps reported here are not a contamination artifact.

Out of scope (phase-3)

- ▶ Cross-benchmark generalisation (PlanBench) and comparison to published SOTA formalizer baselines (e.g. Huang & Zhang, ACL 2025) are deliberately OUT OF SCOPE for this single-tool-use deck.
- ▶ This deck answers: within our 5-model $\times$ 5-task matrix, does giving the model a planning/validation tool (and steering it to use the tool) raise success, and does that advantage track difficulty?

Backup — detailed tables

The slides that follow hold the full per-model numbers behind the token section:

- ▶ total tokens/trial (input+output) per task $\times$ model $\times$ arm, with cost multiples vs no-tools;
- ▶ cost-of-pass (total tokens per success) with bootstrap 95% CIs;
- ▶ the secondary completion-only generation-cost lens — output tokens over successes only. It excludes the re-fed tool input (the dominant tool cost) and failed-attempt tokens, so it flatters tools; it is a generation-length diagnostic, never the consumption headline;
- ▶ the RQ0.6 difficulty-bin table.

Backup — total tokens/trial (input+output) · ≥4B, think=off · 1/2

task	model	no-tools	+tool(plain)	+tool(steered)
solve	Qwen3.5-4B	5.7k (1.2k+4.5k)	25k (22k+2.7k) 4.3× costlier	25k (22k+2.8k) 4.4× costlier
	Qwen3.5-9B	4.9k (1.2k+3.7k)	21k (18k+2.4k) 4.2× costlier	19k (17k+2.3k) 3.9× costlier
	Gemma-MoE-26B	3.8k (1.2k+2.6k)	18k (16k+2.3k) 4.6× costlier	15k (13k+2.0k) 4.0× costlier
	Qwen3.6-35B	4.7k (1.2k+3.5k)	18k (14k+3.7k) 3.7× costlier	20k (17k+3.1k) 4.2× costlier
validate_domain	Qwen3.5-4B	922 (794+128)	10k (9.2k+885) 11.0× costlier	10k (9.2k+891) 11.0× costlier
	Qwen3.5-9B	2.4k (794+1.6k)	10.0k (9.2k+728) 4.2× costlier	10k (9.3k+729) 4.2× costlier
	Gemma-MoE-26B	1.4k (819+624)	9.0k (8.2k+799) 6.2× costlier	9.0k (8.2k+795) 6.2× costlier
	Qwen3.6-35B	2.1k (794+1.3k)	10k (9.5k+849) 4.9× costlier	10k (9.3k+748) 4.7× costlier
validate_problem	Qwen3.5-4B	1.3k (1.1k+114)	19k (16k+3.2k) 15.0× costlier	19k (15k+3.3k) 14.9× costlier
	Qwen3.5-9B	2.1k (1.1k+940)	13k (12k+1.3k) 6.3× costlier	13k (12k+1.4k) 6.3× costlier
	Gemma-MoE-26B	1.9k (1.2k+710)	11k (9.8k+1.2k) 5.8× costlier	11k (9.7k+1.2k) 5.7× costlier
	Qwen3.6-35B	2.1k (1.1k+962)	12k (11k+1.5k) 5.9× costlier	12k (11k+1.2k) 5.6× costlier

Backup — total tokens/trial (input+output) · ≥4B, think=off · 2/2

task	model	no-tools	+tool(plain)	+tool(steered)
validate_plan	Qwen3.5-4B	3.0k (1.4k+1.6k)	31k (27k+4.1k) 10.3× costlier	26k (22k+3.6k) 8.7× costlier
	Qwen3.5-9B	3.1k (1.4k+1.8k)	14k (12k+1.5k) 4.5× costlier	14k (12k+1.4k) 4.4× costlier
	Gemma-MoE-26B	2.5k (1.4k+1.1k)	6.5k (5.3k+1.2k) 2.6× costlier	12k (10k+1.4k) 4.6× costlier
	Qwen3.6-35B	3.4k (1.4k+2.0k)	13k (11k+1.6k) 3.8× costlier	14k (12k+1.4k) 4.2× costlier
simulate	Qwen3.5-4B	5.3k (1.4k+3.9k)	26k (23k+3.3k) 4.9× costlier	30k (26k+3.7k) 5.6× costlier
	Qwen3.5-9B	4.6k (1.4k+3.2k)	18k (15k+2.2k) 3.8× costlier	20k (18k+2.0k) 4.3× costlier
	Gemma-MoE-26B	4.8k (1.5k+3.3k)	17k (15k+1.8k) 3.5× costlier	17k (16k+1.8k) 3.6× costlier
	Qwen3.6-35B	4.0k (1.4k+2.6k)	15k (13k+2.0k) 3.7× costlier	17k (15k+1.6k) 4.1× costlier

Backup — cost-of-pass, tokens per success [bootstrap 95% CI] · ≥4B, think=off · 1/2

task	model	no-tools	+tool(plain)	+tool(steered)
solve	Qwen3.5-4B	74k [53k,120k]	32k [29k,36k] 0.4× cheaper	31k [29k,34k] 0.4× cheaper
	Qwen3.5-9B	46k [34k,68k]	21k [20k,22k] 0.5× cheaper	19k [18k,20k] 0.4× cheaper
	Gemma-MoE-26B	50k [35k,82k]	18k [17k,19k] 0.4× cheaper	16k [15k,16k] 0.3× cheaper
	Qwen3.6-35B	50k [37k,77k]	28k [26k,30k] 0.6× cheaper	21k [20k,23k] 0.4× cheaper
validate_domain	Qwen3.5-4B	4.8k [3.9k,6.2k]	11k [10k,11k] 2.2× costlier	11k [10k,11k] 2.2× costlier
	Qwen3.5-9B	9.3k [7.6k,12k]	10.0k [9.8k,10k] 1.1× costlier	10k [9.9k,10k] 1.1× costlier
	Gemma-MoE-26B	1.9k [1.7k,2.0k]	9.2k [9.0k,9.5k] 5.0× costlier	9.2k [9.0k,9.4k] 4.9× costlier
	Qwen3.6-35B	3.1k [2.9k,3.5k]	10k [10k,11k] 3.3× costlier	10k [9.9k,10k] 3.2× costlier
validate_problem	Qwen3.5-4B	2.2k [2.1k,2.4k]	24k [22k,26k] 10.7× costlier	23k [21k,25k] 10.3× costlier
	Qwen3.5-9B	3.2k [2.9k,3.5k]	14k [13k,14k] 4.3× costlier	15k [14k,15k] 4.6× costlier
	Gemma-MoE-26B	2.5k [2.4k,2.7k]	11k [11k,11k] 4.4× costlier	11k [11k,11k] 4.3× costlier
	Qwen3.6-35B	2.8k [2.6k,3.0k]	13k [12k,13k] 4.6× costlier	12k [12k,12k] 4.4× costlier

Backup — cost-of-pass, tokens per success [bootstrap 95% CI] · ≥4B, think=off · 2/2

task	model	no-tools	+tool(plain)	+tool(steered)
validate_plan	Qwen3.5-4B	4.1k [3.9k,4.2k]	44k [43k,46k] 10.9× costlier	33k [32k,34k] 8.1× costlier
	Qwen3.5-9B	3.9k [3.8k,4.0k]	15k [15k,15k] 3.8× costlier	14k [14k,14k] 3.6× costlier
	Gemma-MoE-26B	2.9k [2.8k,3.0k]	32k [30k,34k] 11.0× costlier	13k [12k,13k] 4.3× costlier
	Qwen3.6-35B	3.7k [3.6k,3.8k]	16k [15k,16k] 4.2× costlier	14k [14k,14k] 3.8× costlier
simulate	Qwen3.5-4B	— (0 succ)	43k [38k,50k]	51k [45k,60k]
	Qwen3.5-9B	— (0 succ)	27k [25k,30k]	24k [22k,26k]
	Gemma-MoE-26B	— (0 succ)	19k [17k,20k]	19k [18k,21k]
	Qwen3.6-35B	— (0 succ)	20k [19k,21k]	17k [16k,18k]

Backup — completion-only output tokens per success (secondary, tool-flattering) • 1/2

task	model	no-tools	+tool(plain)	+tool(steered)
solve	Qwen3.5-4B	2.8k [1.7k,3.9k]	2.4k [2.2k,2.6k]	2.5k [2.4k,2.7k]
	Qwen3.5-9B	1.9k [1.3k,2.6k]	2.4k [2.3k,2.6k]	2.3k [2.1k,2.4k]
	Gemma-MoE-26B	1.2k [575,2.1k]	2.3k [2.2k,2.4k]	2.0k [1.8k,2.1k]
	Qwen3.6-35B	2.2k [1.8k,2.7k]	3.2k [3.0k,3.4k]	2.9k [2.8k,3.1k]
validate_domain	Qwen3.5-4B	95 [45,155]	863 [772,961]	886 [785,996]
	Qwen3.5-9B	865 [694,1.1k]	728 [692,765]	729 [694,765]
	Gemma-MoE-26B	567 [544,594]	797 [760,836]	794 [756,832]
	Qwen3.6-35B	1.2k [1.1k,1.3k]	827 [767,892]	748 [711,786]
validate_problem	Qwen3.5-4B	108 [76,150]	2.5k [2.3k,2.8k]	2.7k [2.5k,3.0k]
	Qwen3.5-9B	619 [558,687]	1.3k [1.3k,1.4k]	1.4k [1.3k,1.4k]
	Gemma-MoE-26B	572 [555,589]	1.2k [1.2k,1.3k]	1.2k [1.2k,1.2k]
	Qwen3.6-35B	892 [840,947]	1.5k [1.4k,1.6k]	1.2k [1.1k,1.3k]

Backup — completion-only output tokens per success (secondary, tool-flattering) • 2/2

task	model	no-tools	+tool(plain)	+tool(steered)
validate_plan	Qwen3.5-4B	1.5k [1.5k,1.6k]	3.4k [3.3k,3.5k]	2.9k [2.9k,3.0k]
	Qwen3.5-9B	1.6k [1.6k,1.7k]	1.4k [1.4k,1.4k]	1.4k [1.3k,1.4k]
	Gemma-MoE-26B	1.1k [1.1k,1.1k]	1.0k [1.0k,1.1k]	1.4k [1.4k,1.4k]
	Qwen3.6-35B	1.9k [1.9k,2.0k]	1.6k [1.6k,1.6k]	1.4k [1.4k,1.5k]
simulate	Qwen3.5-4B	— (0 succ)	2.0k [1.8k,2.2k]	2.4k [2.1k,2.7k]
	Qwen3.5-9B	— (0 succ)	1.8k [1.7k,1.9k]	1.8k [1.6k,1.9k]
	Gemma-MoE-26B	— (0 succ)	1.5k [1.5k,1.6k]	1.6k [1.5k,1.7k]
	Qwen3.6-35B	— (0 succ)	1.7k [1.7k,1.8k]	1.5k [1.5k,1.6k]

Backup — RQ0.6 difficulty-binned success (object count)

task	bin	no-tools	+tool(steered)	Δ (pp)	n/arm
solve	≤ 6	14 [10,18]	100 [99,100]	+86	351
	6–14	8 [5,12]	97 [94,98]	+89	261
	> 14	5 [3,8]	93 [90,96]	+89	288
validate_problem	≤ 5	73 [69,76]	94 [92,96]	+22	576
	5–13	73 [69,76]	96 [94,97]	+23	603
	> 13	74 [71,78]	97 [95,98]	+23	540
validate_plan	≤ 6	83 [81,85]	96 [95,97]	+13	1755
	6–14	86 [84,87]	97 [96,98]	+12	1305
	> 14	79 [77,81]	93 [92,94]	+14	1440
simulate	≤ 6	0 [0,1]	89 [86,92]	+89	351
	6–14	0 [0,1]	93 [89,95]	+93	261
	> 14	0 [0,1]	89 [85,92]	+89	288